

Task 2

Synthesis & Gate-Level Simulation

8-bit Arithmetic Logic Unit (alu_8bit)

Toolchain: Yosys 0.52 • Icarus Verilog 12.0 • WSL2 / Ubuntu

1. Goal

Synthesize the RTL description of the 8-bit ALU (alu_8bit.v) into a gate-level netlist using an open-source synthesis tool, then perform gate-level simulation (GLS) and confirm that gate-level behavior is functionally equivalent to the original RTL behavior.

2. Tools & Environment

Item	Detail
Operating environment	Windows 11 with WSL2 (Ubuntu)
Synthesis tool	Yosys 0.52 (open-source)
Simulation tool	Icarus Verilog 12.0 (iverilog / vvp)
Waveform viewer	GTKWave
Design under test	alu_8bit.v — 8-bit ALU, 12 opcodes, combinational
Testbench	tb_alu_8bit.v — self-checking, 21 directed + 200 random tests

3. Step 1 — RTL Simulation (Baseline)

Before synthesis, the original RTL was simulated against the testbench to establish a baseline of correct behavior.

Commands

```
iverilog -o rtl_sim alu_8bit.v tb_alu_8bit.v
vvp rtl_sim > rtl_output.log
tail -20 rtl_output.log
```

RTL Simulation Result

```
=====
SUMMARY
=====
Total checks run : 221
Failures        : 0
```

```
Opcode coverage : 12 / 12 opcodes hit (mask=111111111111)
RESULT: ALL TESTS PASSED
```

All 221 checks (21 directed test cases covering every opcode and edge condition, plus 200 randomized vectors) passed with zero failures. This log (rtl_output.log) serves as the reference baseline for equivalence checking after synthesis.

4. Step 2 — Synthesis (RTL → Gate-Level Netlist)

The design was synthesized using Yosys with a generic logic-gate mapping (AND, OR, XOR, NOT), suitable for an academic/open-source synthesis flow since alu_8bit is a purely combinational design with no internal sequential elements.

Synthesis Script — synth.js

```
read_verilog alu_8bit.v
hierarchy -check -top alu_8bit
proc
opt
techmap
opt
abc -g AND,OR,XOR
opt_clean
write_verilog -noattr netlist.v
stat
```

Command

```
yosys synth.js
```

Synthesis Result — Gate Count (from stat report)

Cell Type	Count
\$_AND_	190
\$_OR_	194
\$_NOT_	62
\$_XOR_	22
Total cells	468

Synthesis completed without errors. Yosys reduced the design from behavioral Verilog (case statements, arithmetic operators) down to 468 primitive logic gates, mapped via the ABC logic optimizer. The resulting gate-level netlist was written to netlist.v, preserving the original module name and port list (a, b, opcode, result, carry, overflow, zero, negative) so it remains a drop-in replacement for the RTL module in simulation.

5. Step 3 — Gate-Level Simulation

The synthesized netlist was simulated using the same self-checking testbench used for the RTL. Since the netlist contains Yosys's internal primitive cells, Yosys's own simulation model library (simlib.v) was included at compile time to define their behavior for Icarus Verilog.

Commands

```
iverilog -o gls_sim netlist.v tb_alu_8bit.v /usr/share/yosys/simlib.v
vvp gls_sim > gls_output.log
tail -20 gls_output.log
```

Gate-Level Simulation Result

```
=====
SUMMARY
=====
Total checks run : 221
Failures       : 0
Opcode coverage : 12 / 12 opcodes hit (mask=111111111111)
RESULT: ALL TESTS PASSED
```

The gate-level netlist passed all 221 checks across all 12 opcodes — an identical pass count to the RTL simulation.

6. RTL vs Gate-Level Equivalence Check

To formally confirm that gate-level behavior matches RTL behavior, the two simulation logs were directly compared using a file diff.

Command

```
echo "Diff between RTL and GLS output: " && diff rtl_output.log gls_output.log &&
echo "IDENTICAL - 0 differences found"
```

Result

```
Diff between RTL and GLS output:
IDENTICAL - 0 differences found
```

The diff produced zero output before the confirmation message printed, meaning rtl_output.log and gls_output.log are byte-for-byte identical. This formally confirms that the synthesized gate-level netlist is functionally equivalent to the original RTL across all 221 test vectors and all 12 ALU operations.

7. Deliverables Summary

Deliverable	File(s)
1. Synthesis script & generated netlist	synth.ys, netlist.v
2. Simulation logs showing equivalence	rtl_output.log, gls_output.log

8. Conclusion

The 8-bit ALU was successfully synthesized from RTL to a 468-gate netlist using Yosys, and gate-level simulation using Icarus Verilog confirmed exact functional equivalence with the original RTL across all directed and randomized test vectors. No discrepancies were found between RTL and gate-level behavior, satisfying both the synthesis and verification requirements of this task.